Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ingeniería en Sistemas de Información

Entregable 2: Documento de Architecture Decision Records (ADRs)
Documento anexo al informe final

[Nombre del integrante 1] [Carné]
[Nombre del integrante 2] [Carné]
[Nombre del integrante 3] [Carné]

Arquitectura de Computadores II, sección 040
[Nombre del catedrático]
31 de octubre de 2026

Índice

Documento de decisiones de arquitectura (ADRs) 1

Propósito del documento 1

Cliente hipotético 1

Resumen de decisiones 2

Decisiones obligatorias 2

ADR-01. Plataforma de despliegue 2

ADR-02. Broker de eventos 5

ADR-03. Persistencia de eventos 6

ADR-04. Orquestación de tareas periódicas 8

ADR-05. Contenerización y orquestación 9

ADR-06. Resiliencia y tolerancia a fallos 11

ADR-07. Seguridad y privacidad de datos 13

ADR-08. Observabilidad y SLOs 14

Decisiones adicionales 16

ADR-09. Diseño de topics y clave de partición 16

ADR-10. Modelo de correlación e inferencia 17

ADR-11. Lenguaje de implementación 19

ADR-12. Formato de eventos 20

Referencias 22

Documento de decisiones de arquitectura (ADRs)

Propósito del documento

Este documento, anexo al informe final, reúne las decisiones de arquitectura del sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez. Cada decisión se registra como un Architecture Decision Record (ADR), un formato breve propuesto por Nygard (2011) para dejar por escrito el problema que obligó a decidir, las alternativas evaluadas, la elección y sus consecuencias, de modo que cualquier persona pueda entender después por qué el sistema es como es.

Todos los ADRs siguen el formato obligatorio de la sección 4.1 del enunciado: contexto, opciones consideradas (mínimo dos alternativas reales con sus ventajas y desventajas), decisión, consecuencias e impacto en el negocio redactado para un directivo no técnico. Además, cuando existe, se agrega la evidencia medida en las pruebas del sistema, para que cada decisión se respalde con números y no solo con argumentos.

Cliente hipotético

El cliente es la Municipalidad de San Juan Sacatepéquez, departamento de Guatemala: un municipio mediano de 218 156 habitantes (Instituto Nacional de Estadística [INE], 2019), con presupuesto limitado, sin un equipo grande de tecnología y con la obligación de proteger los datos personales de sus vecinos. El sistema cubre siete sectores del municipio (Casco urbano, Sajcavillá, Cruz Blanca, Lo de Mejía, Ciudad Quetzal, Villas del Quetzal y Montúfar). Esta premisa guía todas las decisiones: se prefiere lo simple, barato de operar y bajo control municipal, siempre que cumpla los objetivos de servicio.

Resumen de decisiones

Tabla 1

Resumen de los ADRs del proyecto

ADR	Tema	Decisión
ADR-01	Plataforma de despliegue	On-premise en un solo servidor con contenedores
ADR-02	Broker de eventos	Apache Kafka 4.3 en modo KRaft
ADR-03	Persistencia de eventos	PostgreSQL 16 + Redis 7.4 (estado) + Elasticsearch 8.19 (búsqueda)
ADR-04	Orquestación de tareas periódicas	Apache Airflow 3.3 en modo standalone
ADR-05	Contenerización y orquestación	Docker Compose con perfiles; sin Kubernetes
ADR-06	Resiliencia y tolerancia a fallos	Dead-letter, reintentos, entrega al menos una vez y deduplicación
ADR-07	Seguridad y privacidad	HMAC de placas, rechazo de placa en claro, retención y mínimo privilegio
ADR-08	Observabilidad y SLOs	Latencia p95 < 2 s y < 2 % de inválidos, con registro de incumplimientos
ADR-09	Diseño de topics	Un topic de eventos estandarizados con clave = sector
ADR-10	Modelo de correlación e inferencia	Reglas determinísticas con ventana de tiempo en Redis
ADR-11	Lenguaje de implementación	Python para todos los servicios
ADR-12	Formato de eventos	JSON con schemaVersion validado con Pydantic

Decisiones obligatorias

ADR-01. Plataforma de despliegue

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

El sistema debe recibir eventos de sensores, correlacionarlos y avisar a las instituciones en pocos segundos. La municipalidad tiene presupuesto limitado, no cuenta con un equipo de operación en la nube y maneja datos sensibles (placas y ubicaciones de vecinos). El enunciado pide elegir entre un modelo on-premise, híbrido o cloud-native, con un costo estimado y la latencia esperada de extremo a extremo.

El volumen normal estimado es de unos 5 eventos por segundo en todo el municipio, con picos de 20 veces ese valor en emergencias o pruebas de carga.

Opciones consideradas

Tabla 2

Opciones de plataforma de despliegue (ADR-01)

Opción	Ventajas	Desventajas
On-premise / LAN (un servidor propio)	Costo fijo y bajo; los datos no salen de la municipalidad; latencia baja y predecible; no depende del internet para recibir y procesar alertas	Sin elasticidad ante picos; un solo servidor es un punto único de falla; la municipalidad mantiene el equipo
Híbrido (ingesta local + analítica en la nube)	Ingesta cercana a los sensores; analítica y respaldo elásticos en la nube	Costo de transferencia de datos (egress); dos entornos que operar e integrar; los datos personales salen del municipio
Cloud-native (servicios gestionados)	Elasticidad; alta disponibilidad gestionada por el proveedor; sin hardware propio	Costo mensual variable y permanente; dependencia del proveedor (vendor lock-in) y del enlace a internet; latencia adicional desde los sensores

Decisión

Se elige **on-premise en un solo servidor**: todo el sistema corre en contenedores dentro de un servidor propio de la municipalidad (en el prototipo, una laptop con Intel i7-13650HX y 32 GB de RAM, de los cuales se asignan 20 GB y 12 núcleos a Docker). Aunque comparten máquina, cada servicio se comunica con los demás solo por red, de modo que la arquitectura sigue siendo distribuida a nivel lógico: si la municipalidad compra un segundo servidor, los contenedores se reparten sin cambiar código.

Consecuencias

Positivas:

- Costo bajo y predecible, sin factura mensual de nube
- Control total de los datos personales dentro de la municipalidad
- Latencia baja y estable, sin depender de la calidad del internet
- Puesta en marcha con un solo comando sobre la configuración versionada

Negativas (aceptadas):

- Sin elasticidad automática: un pico mayor a la capacidad medida se procesa con retraso (no se pierde)
- El servidor es un punto único de falla; se mitiga con UPS, respaldos y un segundo equipo en el futuro
- La municipalidad es responsable del mantenimiento del equipo

Impacto en el negocio

Costo (orden de magnitud, no es cotización): el prototipo usa una laptop ya disponible (unos Q12 000 a Q15 000). Para producción se recomienda un servidor de 8 a 16 núcleos, 32 a 64 GB de RAM y discos en espejo, más una UPS: entre Q28 000 y Q45 000 de inversión única, más unos Q200 mensuales de electricidad. Todo el software es de código abierto, sin licencias. Un equivalente con servicios gestionados en la nube (Kafka, PostgreSQL, Redis, búsqueda, Airflow y cómputo para los contenedores) se estima en US$900 a US$1 300 al mes según precios públicos de lista, es decir, unos Q84 000 a Q120 000 al año: la inversión on-premise se recupera en menos de seis meses.

Riesgo: el principal es la falla del servidor; se acepta para esta escala y se mitiga con respaldos diarios de la base de datos y la opción de agregar un segundo servidor. **Tiempo de implementación:** el sistema completo se levanta en menos de una hora siguiendo el manual (README).

Latencia esperada de extremo a extremo: menos de 2 segundos (objetivo del ADR-08). Medido: entre 30 y 200 ms con tráfico normal y p95 de 976 ms con 508 eventos por segundo sostenidos.

Evidencia medida

- Consumo bajo carga máxima sostenida: unos 4.6 GB de 20 GB de RAM y menos de 1 núcleo de 12
- Capacidad medida: unos 510 eventos/s de extremo a extremo dentro del SLO, unas 100 veces el tráfico normal

ADR-02. Broker de eventos

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Los sensores producen eventos a ritmos distintos y el correlador, el despachador y otros consumidores deben leerlos sin perder ninguno, en orden por sector y aunque alguno de ellos esté caído por un rato. Se necesita un intermediario (broker) que desacople a quienes producen de quienes consumen.

Opciones consideradas

Tabla 3

Opciones de broker de eventos (ADR-02)

Opción	Ventajas	Desventajas
Apache Kafka	Registro persistente y ordenado por partición; los mensajes quedan guardados aunque ya se hayan leído (permite reprocesar); alto throughput; varios grupos de consumidores leen lo mismo	Más complejo de operar que una cola simple; consume más memoria
RabbitMQ	Simple de operar; enrutamiento flexible; buena latencia	Modelo de cola: el mensaje se borra al consumirse, lo que dificulta reprocesar el historial; menor throughput sostenido
Servicio gestionado (Amazon SQS, Google Pub/Sub)	Sin operación propia; escala solo	Requiere nube (contradice el ADR-01); costo por mensaje; dependencia del internet; latencia mayor

Decisión

Se elige **Apache Kafka 4.3** (Kreps et al., 2011) en modo KRaft, es decir, sin Zookeeper: un solo contenedor que es a la vez broker y controlador. Los topics los crea un contenedor de inicialización (`kafka-init`) y está desactivada la creación automática de topics. La retención es de 7 días y los productores usan `acks=all` con idempotencia.

Consecuencias

Positivas:

- Si el correlador se cae, los eventos esperan guardados en Kafka y se procesan al volver

- El orden por sector se garantiza con la clave de partición (ADR-09)
- El correlador y el despachador leen los mismos datos sin interferirse (grupos de consumidores distintos)
- La ingestión medida supera los 26 500 eventos/s

Negativas (aceptadas):

- Un solo broker significa factor de replicación 1: si se daña el disco, se pierde el historial de Kafka (riesgo aceptado del ADR-01)
- Kafka es el único componente cuya caída prolongada puede perder eventos (ver ADR-06)

Impacto en el negocio

Costo: cero en licencias; unos 450 MB de RAM del servidor. **Riesgo:** bajo para la escala municipal; la caída de Kafka se detecta de inmediato porque los ingestores responden HTTP 503. **Tiempo:** configuración incluida en el despliegue estándar, sin trabajo adicional.

Evidencia medida

- Kafka caído 32 s con tráfico x20: 0 eventos perdidos; caído 70 s: unos 300 perdidos de 13 215 (2.3 %)
- Correlador detenido con 15 000 eventos en cola: Kafka los conservó todos y se procesaron al volver

ADR-03. Persistencia de eventos

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Los eventos son datos con marca de tiempo y formatos distintos según el sensor. Se necesita guardar eventos, alertas, envíos a instituciones y estadísticas para auditoría y visualización, mantener un estado de corto plazo muy rápido para correlacionar y permitir búsquedas históricas de texto. El volumen normal es de unos 430 000 eventos por día.

Opciones consideradas

Tabla 4

Opciones de persistencia (ADR-03)

Opción	Ventajas	Desventajas
PostgreSQL (relacional) con JSONB	Confiable y conocido; transacciones; uniones entre eventos, alertas y sectores; JSONB guarda formatos distintos sin una tabla por sensor; Grafana lo lee directo	No está optimizado para series de tiempo enormes
Base de series de tiempo (TimescaleDB, InfluxDB)	Muy eficiente para métricas en el tiempo y agregaciones por intervalo	InfluxDB no maneja bien las relaciones entre alertas y eventos; TimescaleDB agrega una extensión más que mantener; el volumen actual no lo necesita
NoSQL (MongoDB, Cassandra)	Esquema flexible; escala horizontal	Consultas de auditoría con uniones más difíciles; consistencia más débil; otro motor que aprender

Decisión

Se usa **PostgreSQL 16** como almacén principal (eventos con `data` y `metadata` en JSONB, alertas, envíos, inválidos, violaciones de SLO y estadísticas, todo con `TIMESTAMPTZ` e índices por tiempo y sector), **Redis 7.4** con persistencia AOF como memoria de corto plazo del correlador y **Elasticsearch 8.19** solo para la búsqueda histórica de alertas (perfil completo).

Consecuencias

Positivas:

- Un solo motor relacional cubre auditoría, trazabilidad (eventId y sourceEventIds) y visualización
- Llaves foráneas: la base rechaza un sector inexistente aunque fallara la validación
- Redis responde en microsegundos para las consultas por evento del correlador
- Elasticsearch permite búsquedas de texto sin cargar la base principal

Negativas (aceptadas):

- Tres motores de datos que respaldar y vigilar

- Con un volumen 100 veces mayor sostenido convendría particionar la tabla de eventos o evaluar TimescaleDB

Impacto en el negocio

Costo: cero en licencias; unos 240 MB (PostgreSQL), 180 MB (Redis) y 1.6 GB (Elasticsearch) de RAM bajo carga. **Riesgo:** bajo; tecnologías maduras y documentadas. **Tiempo:** el esquema se crea solo al iniciar el contenedor.

Evidencia medida

- 448 eventos aceptados = 448 en PostgreSQL tras reiniciar el correlador a mitad de tráfico
- 200 alertas en PostgreSQL = 200 documentos en Elasticsearch al cargar el histórico

ADR-04. Orquestación de tareas periódicas

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Hay tareas que no son en tiempo real: borrar datos vencidos (privacidad), resumir alertas por día y calcular la proporción de falsos positivos por sector. El enunciado pregunta si se justifica la complejidad operativa de Airflow a esta escala.

Opciones consideradas

Tabla 5

Opciones de orquestación de tareas (ADR-04)

Opción	Ventajas	Desventajas
Apache Airflow (standalone)	Interfaz web con historial, reintentos y ejecución manual; estándar de la industria; pedido por el enunciado	Pesado: unos 1.3 GB de RAM para tres tareas SQL; más piezas internas
Scheduler liviano (cron, APScheduler)	Muy liviano (unos 30 MB); trivial de configurar	Sin interfaz ni historial de ejecuciones; reintentos y monitoreo a mano

Decisión

Se usa **Apache Airflow 3.3 en modo standalone** (un contenedor con scheduler, servidor web y procesador de DAGs, metadatos en SQLite) con tres DAGs

que ejecutan SQL directo: `retencion_datos` (diario), `reporte_diario` (diario y manual, recalcula sin duplicar) y `analisis_falsos_positivos` (cada hora). La respuesta honesta a la pregunta del enunciado es que, a esta escala, Airflow **no** se justifica por carga; se justifica por la trazabilidad de las ejecuciones (quién corrió qué y cuándo), la ejecución manual desde la web durante la demostración y porque deja preparado el crecimiento a más tareas de análisis.

Consecuencias

Positivas:

- Historial visible de cada ejecución y su resultado
- Ejecución manual de los reportes desde la interfaz
- Las tareas pueden crecer sin cambiar de herramienta

Negativas (aceptadas):

- Es el servicio que más memoria usa en reposo (1.33 GB)
- SQLite como metadatos no es apto para producción; en producción iría con PostgreSQL

Impacto en el negocio

Costo: cero en licencias; 1.3 GB de RAM del servidor (holgado dentro de los 20 GB). **Riesgo:** bajo; si Airflow falla, el monitoreo en tiempo real sigue funcionando porque no depende de él. **Tiempo:** los tres DAGs se cargan solos al iniciar.

Evidencia medida

- Las tres ejecuciones manuales terminaron en estado success, con 0 errores de importación
- Solo está activo en el perfil completo (full); el perfil de desarrollo (core) no lo levanta

ADR-05. Contenerización y orquestación

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

El perfil completo tiene 15 servicios que deben levantarse en orden (Kafka y la base de datos antes que quienes los usan), con límites de memoria y que se recuperen

solos si fallan. Debe poder instalarse igual en cualquier máquina del grupo y en el laboratorio.

Opciones consideradas

Tabla 6

Opciones de contenerización (ADR-05)

Opción	Ventajas	Desventajas
Docker Compose	Un archivo describe todo; arranque con un comando; perfiles (core y full); healthchecks y orden de arranque; bajo consumo	Un solo nodo; sin autoescalado ni conmutación por falla entre máquinas
Kubernetes	Autoescalado, despliegues sin corte, conmutación por falla entre nodos (Burns et al., 2016)	Plano de control propio (más memoria y complejidad); muchos conceptos nuevos; en un solo servidor no da alta disponibilidad real

Decisión

Se usa **Docker Compose** (Merkel, 2014) con dos perfiles: `core` para desarrollo y `full` para la demostración (agrega Elasticsearch y Airflow). Se decide conscientemente **no usar Kubernetes**: en un solo servidor agrega complejidad sin ganar disponibilidad, porque si la máquina se apaga, todo se cae igual. Cada servicio tiene límite de memoria, healthcheck cuando aplica, dependencias por estado sano y reinicio automático.

Consecuencias

Positivas:

- Instalación reproducible: verificada clonando el repositorio en una carpeta nueva y levantando todo solo con el manual
- Imágenes con versiones fijas (sin latest): el sistema no cambia solo
- Si un servicio falla, Docker lo reinicia

Negativas (aceptadas):

- Sin autoescalado: la capacidad está fijada por el servidor
- Migrar a varios servidores requeriría Docker Swarm o Kubernetes

Impacto en el negocio

Costo: cero; Docker Desktop y Docker Engine son gratuitos para este uso. **Riesgo:** bajo; la configuración está versionada en git. **Tiempo:** instalación de cero a sistema funcionando en menos de una hora.

Evidencia medida

- Perfil completo bajo carga: unos 4.6 GB en total; los servicios Python usan menos de 60 MB cada uno
- Suma de los límites de memoria: unos 12.4 GB, dentro de los 20 GB asignados

ADR-06. Resiliencia y tolerancia a fallos

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Cualquier componente puede caerse durante la operación o la demostración: un ingestor, el correlador, Redis o el propio broker. El enunciado pide definir el manejo de caídas del broker, colas de mensajes muertos (dead-letter queue), reintentos y la pérdida aceptable de eventos.

Opciones consideradas

Tabla 7

Opciones de manejo de fallos (ADR-06)

Opción	Ventajas	Desventajas
Entrega al menos una vez + deduplicación	No se pierde nada ante caídas de consumidores; duplicados controlados por eventId	Puede haber trabajo repetido que hay que filtrar
Entrega como máximo una vez	Simple; nunca hay duplicados	Una caída pierde los eventos en proceso: inaceptable para alertas de seguridad
Exactamente una vez (transacciones de Kafka)	Sin duplicados ni pérdidas dentro de Kafka	Mucho más complejo; no cubre escrituras externas (PostgreSQL, Redis, REST) sin coordinación adicional

Decisión

Se adopta **entrega al menos una vez con deduplicación**: el correlador confirma el offset de Kafka solo después de guardar en PostgreSQL, publicar las alertas y marcar los eventos en Redis; si se cae en medio, reprocesa el lote y descarta lo ya hecho por eventId. Además: NGINX saca de servicio un ingestor que falla y reintenta en otro; los eventos inválidos van al topic `events.invalid` (dead-letter) con su motivo; los ingestores responden HTTP 503 si Kafka no confirma en 5 s y el cliente reintenta con espera creciente (1, 2, 4 y 8 s, hasta 5 intentos); el correlador se apaga en orden ante SIGTERM y Docker reinicia cualquier servicio caído. **Pérdida aceptable:** cero ante la caída de cualquier componente distinto de Kafka; ante una caída de Kafka, el cliente aguanta unos 40 s por lote y más allá la pérdida es proporcional a la duración de la caída.

Consecuencias

Positivas:

- Medido: 0 eventos perdidos y 0 alertas duplicadas al caer un ingestor, el correlador o Redis
- Ningún evento inválido se descarta en silencio

Negativas (aceptadas):

- Una caída de Kafka mayor a unos 40 s pierde eventos (2.3 % en una prueba de 70 s)
- Para eliminar esa pérdida, los sensores necesitarían guardar y reenviar (store and forward)

Impacto en el negocio

Costo: ninguno adicional; son decisiones de diseño. **Riesgo:** acotado y medido; el único escenario con pérdida es una caída prolongada del broker, que se detecta de inmediato. **Tiempo:** incluido en el desarrollo.

Evidencia medida

- Caída de ingestor-2 por 20 s: 9003 aceptados = 9003 guardados
- Reinicio del correlador: de vuelta en menos de 1 s gracias al apagado ordenado

- Kafka caído 70 s: 251 eventos que el cliente dio por perdidos sí llegaron (quedaron en la cola del productor) y la deduplicación evitó contarlos dos veces

ADR-07. Seguridad y privacidad de datos

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Las placas vehiculares y la ubicación geográfica son datos personales. Aunque en la simulación son sintéticos, el enunciado exige tratarlos como si fueran reales. En Guatemala, la Ley de Acceso a la Información Pública (Congreso de la República de Guatemala, 2008) reconoce la protección de los datos personales en poder de las instituciones públicas, y el Reglamento General de Protección de Datos europeo (Parlamento Europeo y Consejo de la Unión Europea, 2016) es una referencia internacional para la seudonimización.

Opciones consideradas

Tabla 8

Opciones de protección de placas (ADR-07)

Opción	Ventajas	Desventajas
Guardar la placa en claro con control de acceso	Simple; permite ver la placa completa	Cualquier fuga expone datos personales; contradice el principio de minimización
Cifrado reversible de la placa	Se puede recuperar la placa con la clave	La clave debe estar disponible en todos los servicios: una fuga de la clave expone todo
Seudonimización con HMAC-SHA256	Sin la clave no se puede recuperar la placa; dos lecturas de la misma placa dan el mismo hash (permite correlacionar); el operador ve solo una parte	No se puede recuperar la placa completa desde el sistema (se requiere un proceso aparte con la autoridad)

Decisión

Se adopta **seudonimización con HMAC-SHA256** (Krawczyk et al., 1997): la cámara envía solo el hash con la clave `PLATE_HMAC_KEY` y una versión parcial (por ejemplo `P-***BCD`). El ingestor rechaza con el motivo `PLACA_EN_CLARO` cualquier evento que traiga la placa completa. Además: retención de 30 días para eventos y 365 para

alertas (DAG `retencion_datos`); tres usuarios de base de datos con mínimo privilegio (`app`, `grafana_ro` solo lectura y `admin` solo para la creación); contraseñas solo en el archivo `.env`, que no se sube al repositorio; bases de datos publicadas únicamente en 127.0.0.1; y el identificador de usuario de la app es un seudónimo. Elasticsearch corre sin autenticación **solo** por ser laboratorio en una máquina; en producción se activaría su seguridad.

Consecuencias

Positivas:

- Una fuga de la base de datos no revela placas ni identidades
- La correlación de vehículos (R05 y R07) funciona igual con hashes
- Grafana no puede modificar datos aunque alguien edite una consulta

Negativas (aceptadas):

- Recuperar una placa completa para una investigación requiere un proceso fuera del sistema
- Elasticsearch sin autenticación es aceptable solo en laboratorio

Impacto en el negocio

Costo: ninguno. **Riesgo:** reduce el riesgo legal y reputacional de la municipalidad ante una fuga de datos. **Tiempo:** incluido en el desarrollo.

Evidencia medida

- Validado: un evento con placa en claro es rechazado con PLACA_EN_CLARO y queda registrado en el dead-letter
- El usuario grafana_ro recibe permission denied al intentar escribir

ADR-08. Observabilidad y SLOs

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

No basta con tener dashboards: hay que definir umbrales concretos de servicio y qué ocurre cuando se incumplen (Beyer et al., 2016). Una alerta de robo en curso pierde valor si llega tarde, y un porcentaje alto de eventos inválidos indica sensores dañados o mal configurados.

Opciones consideradas

Tabla 9

Opciones de observabilidad (ADR-08)

Opción	Ventajas	Desventajas
SLOs explícitos medidos en el propio flujo	Cada alerta registra su latencia; los incumplimientos quedan en una tabla y en el log; visible en Grafana	Requiere diseñar qué medir desde el inicio
Solo dashboards de métricas generales	Rápido de armar	No define cuándo algo está mal ni deja evidencia de los incumplimientos

Decisión

Se definen dos SLOs: **(1) latencia de alertas p95 menor a 2 000 ms**, medida como la hora de creación de la alerta menos el `emittedAt` del último evento que completó la regla; **(2) menos de 2 % de eventos inválidos.** Cuando una alerta supera el umbral, el correlador escribe una advertencia en el log y la registra en la tabla `slo_violations`; el dashboard 2 de Grafana muestra el p95 en rojo, la línea del umbral, el porcentaje de inválidos y la lista de violaciones. Los falsos positivos no cuentan para el SLO de latencia porque, por definición, esperan la ventana completa.

Consecuencias

Positivas:

- El incumplimiento queda registrado y visible, no solo se nota
- Permitió encontrar y corregir problemas reales (una espera de 1 s en el correlador y el reparto lento de particiones tras un reinicio)

Negativas (aceptadas):

- Al procesar un atraso tras una caída se registran muchas violaciones; es correcto, porque reflejan lo que vivió el operador

Impacto en el negocio

Costo: ninguno adicional. **Riesgo:** reduce el riesgo de operar a ciegas: la dirección puede ver si el servicio cumple lo prometido. **Tiempo:** incluido en el desarrollo.

Evidencia medida

- Latencia de alertas con tráfico normal: de 30 a 200 ms
- A 508 eventos/s sostenidos: p95 de 976 ms (cumple); a 661 eventos/s: p95 de 13.7 s (incumple, sin pérdida)
- Eventos inválidos: entre 1.4 y 1.5 % (cumple)

Decisiones adicionales

ADR-09. Diseño de topics y clave de partición

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Hay que decidir cómo organizar los mensajes en Kafka. El correlador necesita ver juntos y en orden los eventos de distintos tipos de un mismo sector (por ejemplo, pánico, luego cámara y luego radar en Ciudad Quetzal) para aplicar las reglas.

Opciones consideradas

Tabla 10

Opciones de diseño de topics (ADR-09)

Opción	Ventajas	Desventajas
Un topic por tipo de sensor	Cada tipo tiene su canal; un consumidor puede leer solo el tipo que le interesa	El correlador tendría que leer cinco topics y reordenar los eventos entre ellos: se pierde el orden entre tipos de un mismo sector, que es justo lo que la correlación necesita
Un topic por etapa del flujo, con clave = sector	Todos los eventos de un sector llegan en orden a un solo consumidor; cada etapa (eventos válidos, inválidos, alertas, avisos externos) tiene su propio topic y sus consumidores	Las particiones quedan desbalanceadas porque los sectores tienen tráfico distinto

Decisión

Se usan **seis topics por etapa del flujo**: `events.standardized` (3 particiones, clave = sector), `events.invalid` (dead-letter), `correlated.alerts` (3 particiones) y `external.pnc`, `external.bomberos` y `external.conred` (uno por institución). Separar por

etapa sí organiza a los consumidores: el correlador solo lee eventos válidos, el despachador solo lee alertas e inválidos y cada institución solo lee su propio topic.

Consecuencias

Positivas:

- Orden garantizado por sector para la correlación
- Cada institución recibe únicamente lo que le corresponde

Negativas (aceptadas):

- Desbalance medido: tras 2 minutos de tráfico las particiones tenían 163, 491 y 71 mensajes
- Limita la escalabilidad horizontal del correlador (ver ADR-10)

Impacto en el negocio

Costo: ninguno. **Riesgo:** bajo; el orden por sector evita alertas mal formadas. **Tiempo:** incluido en la configuración inicial.

ADR-10. Modelo de correlación e inferencia

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

El sistema debe inferir situaciones como "posible robo en curso" o "accidente vial" a partir de eventos de varias fuentes, y distinguir los falsos positivos. La lógica debe ser explicable ante actores no técnicos (dirección, área legal), porque una alerta puede movilizar a la PNC.

Opciones consideradas

Tabla 11

Opciones de modelo de correlación (ADR-10)

Opción	Ventajas	Desventajas
Reglas determinísticas con ventana de tiempo (estado en Redis)	Explicables y auditables (cada alerta indica su regla y sus eventos de origen); fáciles de probar con escenarios; sin datos de entrenamiento	Las reglas se mantienen a mano; solo detectan patrones previstos
Motor de procesamiento de eventos complejos (Apache Flink, Esper)	Lenguaje especializado para patrones y ventanas (Luckham, 2002; Etzion & Niblett, 2010); escala a volúmenes grandes	Otra plataforma pesada que operar; curva de aprendizaje alta para el equipo
Aprendizaje automático	Puede descubrir patrones no previstos	Requiere datos históricos reales etiquetados que no existen; difícil de explicar ante el área legal

Decisión

Se implementan **siete reglas determinísticas (R01 a R07) más la regla de falso positivo**, evaluadas por un correlador en Python que mantiene en Redis una ventana de tiempo por sector y tipo de evento (300 s en operación, 60 s en la demostración). Cada alerta lleva un score de 0 a 1 calculado con la confianza de los sensores, la lista de eventos de origen y la regla que la generó. El detalle está en el Entregable 5 (documentación del sistema de correlación e inferencia).

Consecuencias

Positivas:

- Cada alerta es auditable hasta el sensor que la originó
- Probado con 9 escenarios guionados: 9 de 9 alertas esperadas
- Escala horizontalmente porque todo el estado está en Redis

Negativas (aceptadas):

- La regla R01 tal como está definida es sensible al ruido: en tráfico normal casi siempre hay un vehículo saliendo, por lo que en la práctica basta pánico más exceso de velocidad; una mejora futura es exigir que la anomalía sea del mismo vehículo

- Una ventana mayor aumenta las correlaciones por azar

Impacto en el negocio

Costo: ninguno adicional. **Riesgo:** la explicabilidad reduce el riesgo de enviar patrullas por una decisión que nadie puede justificar. **Tiempo:** agregar una regla nueva es una función corta en un solo archivo.

Evidencia medida

- Capacidad del correlador: 660 eventos/s con una instancia y 743 con tres (limitado por la partición más cargada)
- Optimización medida: la primera versión procesaba 66 eventos/s; la versión final, 660

ADR-11. Lenguaje de implementación

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

El diagrama de referencia del catedrático muestra ingestores en Java y Node. El grupo debe poder leer, explicar y modificar todo el código durante la defensa.

Opciones consideradas

Tabla 12

Opciones de lenguaje (ADR-11)

Opción	Ventajas	Desventajas
Varios lenguajes (Java, Node, Python)	Muestra heterogeneidad tecnológica como en el diagrama	Tres ecosistemas que aprender, compilar y mantener; el grupo no domina todos
Un solo lenguaje (Python)	Código legible; librerías maduras (FastAPI, Pydantic, confluent-kafka); todo el grupo puede explicarlo	Rendimiento por proceso menor que Java; el correlador es el cuello de botella medido

Decisión

Se usa **Python 3.12 en todos los servicios** (ingestores, correlador, despachador, aseguradora simulada, simulador y DAGs), con funciones cortas, sin clases innecesarias y comentarios en español. La heterogeneidad que pide el enunciado está en los datos (cinco formatos de sensor), no en los lenguajes.

Consecuencias

Positivas:

- Todo el código es explicable por el grupo
- Cada servicio es autocontenido, con versiones fijas en su requirements.txt

Negativas (aceptadas):

- El correlador en un solo proceso Python limita el máximo de extremo a extremo a unos 510 eventos/s; suficiente para el municipio

Impacto en el negocio

Costo: ninguno. **Riesgo:** reduce la dependencia de una sola persona que conozca un lenguaje. **Tiempo:** menor tiempo de desarrollo y de capacitación.

ADR-12. Formato de eventos

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Cinco tipos de sensores envían datos con estructuras distintas. Se necesita un contrato común que sea fácil de validar, de leer en las pantallas de depuración y de evolucionar en el futuro.

Opciones consideradas

Tabla 13

Opciones de formato de eventos (ADR-12)

Opción	Ventajas	Desventajas
JSON con schemaVersion y validación en el ingestor	Legible a simple vista (Kafka UI, logs); sin servidores adicionales; Pydantic valida tipos y catálogos con poco código	Mensajes más grandes que un formato binario; la compatibilidad se controla a mano
Avro con Schema Registry	Formato binario compacto; compatibilidad de esquemas verificada automáticamente	Un servidor más que operar; los mensajes no se leen a simple vista; mayor complejidad para el grupo

Decisión

Se usa **JSON con el campo schemaVersion** y un sobre común basado en el ejemplo del catedrático (`device`, `location`, `data`, `metadata`), validado con Pydantic en el

ingestor. Los inválidos se rechazan con un motivo legible (por ejemplo `ZONA_DESCONOCIDA` o `TIMESTAMP_EN_EL_FUTURO`).

Consecuencias

Positivas:

- Cualquier persona puede leer un evento en Kafka UI
- El campo schemaVersion permite introducir una versión 2.0 sin romper a los consumidores actuales

Negativas (aceptadas):

- Más bytes por mensaje que Avro; irrelevante al volumen actual

Impacto en el negocio

Costo: ninguno. **Riesgo:** bajo. **Tiempo:** menor que integrar un registro de esquemas.

Referencias

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM, 59*(5), 50–57. https://doi.org/10.1145/2890784

Congreso de la República de Guatemala. (2008). *Ley de Acceso a la Información Pública* (Decreto 57-2008).

Etzion, O., & Niblett, P. (2010). *Event processing in action*. Manning.

Instituto Nacional de Estadística. (2019). *XII Censo Nacional de Población y VII de Vivienda 2018*. https://www.censopoblacion.gt

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*.

Luckham, D. C. (2002). *The power of events: An introduction to complex event processing in distributed enterprise systems*. Addison-Wesley.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Parlamento Europeo y Consejo de la Unión Europea. (2016). *Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales*. Diario Oficial de la Unión Europea, L 119.